Lab Manual

Of

C Programming Lab

F.E. (ALL)

SEM I

F. H. 2026

LIST OF EXPERIMENTS

Lab Outcomes:

After successful completion of the course student will be able to:

- Demonstrate fundamental concepts of C Programming
- Apply the concept of control structures for solving real life problems
- Use modular programming approach for diversified problems
- Represent data in arrays, strings and structures and manipulate them through a program.
- Demonstrate the concept of call by reference, dynamic memory allocation using pointers

Experiment List:

Sr. no.	Experiment	LO Mapping
1.	Introduction to c programming environment.	LO1
2.	Write a program to demonstrate all data types with: • Variable declaration, • Initialization • Input/output statements.	LO1
3.	Using nested if else implement program for largest of 3 numbers	LO2
4.	Given a year, determine whether it is a leap-year or not..	LO2
5.	Menu-Driven code to implement app for billing system	LO2
6.	Write a C Program to print Star pattern - Pyramid star pattern n - Pyramid star pattern • * b) *	LO2
7.	Write a C program to check whether a number is an Armstrong number.	LO2
8.	Write a C program to implement all string operations.	LO2
9.	Write a C of the resultant matrixprogram to find multiplication of two matrix and transpose of the resultant matrix	LO3

10.	Write a C program for sorting numbers using Bubble Sort	LO3
11.	Write a C program for binary search	LO4
12.	Find GCD of two numbers.	LO4
13.	Swap two numbers by call by reference and call by value.	LO3
14.	Write a C program to print Fibonacci series of 20 numbers using recursion	LO3
15.	Find product of digits of a number using recursion.	LO3
16.	Store product details in structure and calculate total amount.	LO3
17.	Demonstrate the example of nested structure.	LO4
18.	Print a string using pointer..	LO4
19.	Demonstrate the example of nested structure.	LO5
20.	Implement program using file handling function.	LO5

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.01

A.1 Aim:

Introduction to c programming environment.

A.2 Outcome:

After successful completion of this experiment students will be,

Demonstrate fundamental concepts of C Programming.

A.3 Theory:

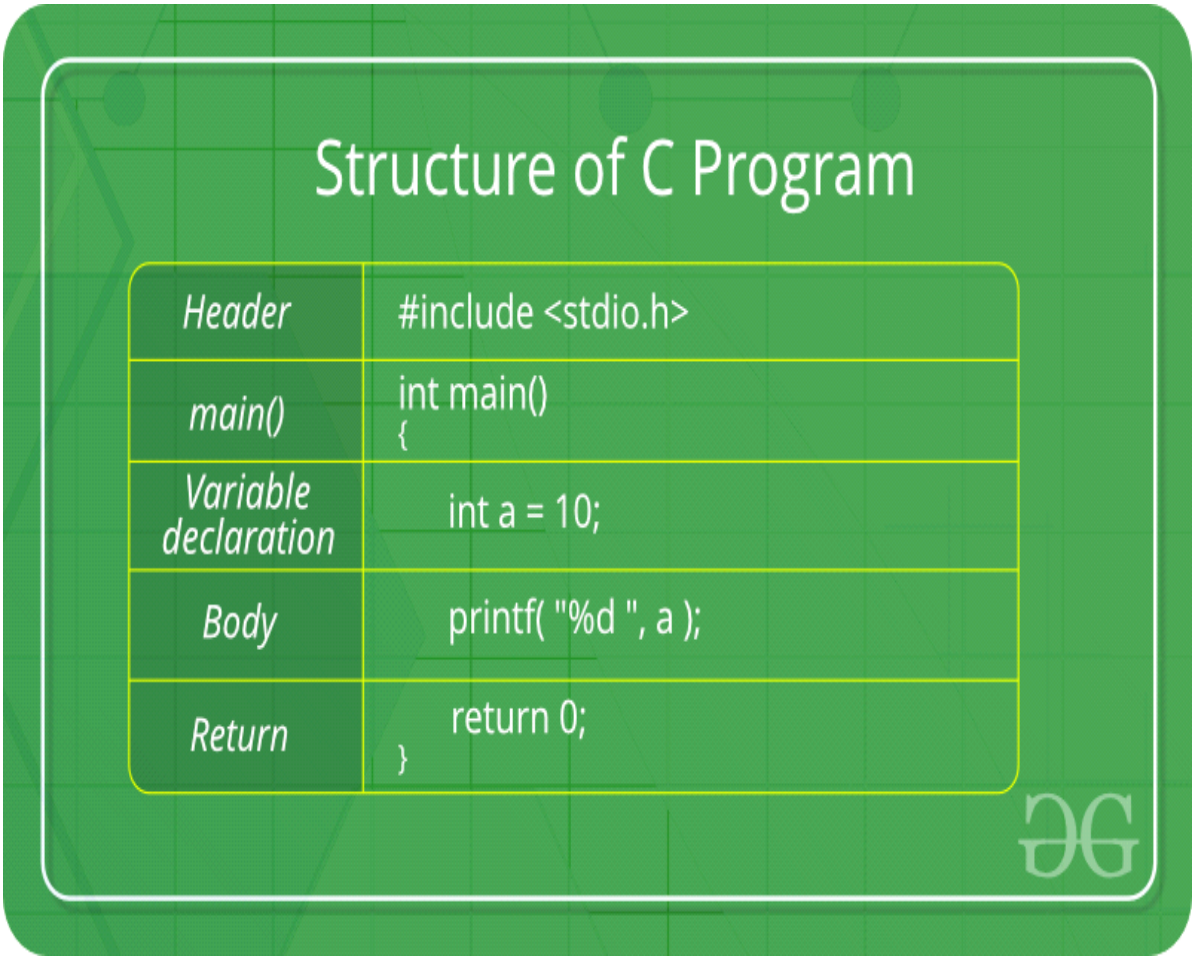
C Language Introduction:

C is a procedural programming language. It was initially developed by Dennis Ritchie in the year 1972. It was mainly developed as a system programming language to write an operating system. The main features of C language include low-level access to memory, a simple set of keywords, and clean style, these features make C language suitable for system programming like an operating system or compiler development.
Many later languages have borrowed syntax/features directly or indirectly from C language. Like syntax of Java, PHP, JavaScript, and many other languages are mainly based on C language. C++ is nearly a superset of C language (There are few programs that may compile in C, but not in C++).

Beginning with C programming:

- **Structure of a C program**
  After the above discussion, we can formally assess the structure of a C program. By structure, it is meant that any program can be written in this structure only. Writing a C program in any other structure will hence lead to a Compilation Error.

  The structure of a C program is as follows:



The components of the above structure are:

- **Header Files Inclusion**: The first and foremost component is the inclusion of the Header files in a C program.
  A header file is a file with extension .h which contains C function declarations and macro definitions to be shared between several source files.

  Some of C Header files:

  - stddef.h – Defines several useful types and macros.
  - stdint.h – Defines exact width integer types.
  - stdio.h – Defines core input and output functions
  - stdlib.h – Defines numeric conversion functions, pseudo-random network generator, memory allocation
  - string.h – Defines string handling functions
  - math.h – Defines common mathematical functions

  Syntax to include a header file in C:

```
#include
```

- **Main Method Declaration:** The next part of a C program is to declare the main() function. The syntax to declare the main function is:

Syntax to Declare main method:

```
int main()
{}
```

- **Variable Declaration:** The next part of any C program is the variable declaration. It refers to the variables that are to be used in the function. Please note that in the C program, no variable can be used without being declared. Also in a C program, the variables are to be declared before any operation in the function.

Example:

```
int main()
{
   int a;
.
.
```

- **Body:** Body of a function in C program, refers to the operations that are performed in the functions. It can be anything like manipulations, searching, sorting, printing, etc.

Example:

```
int main()
{
   int a;

   printf("%d", a);
.
.
```

- **Return Statement:** The last part in any C program is the return statement. The return statement refers to the returning of the values from a function. This return statement and return value depend upon the return type of the function. For example, if the return type is void, then there will be no return statement. In any other case, there will be a return statement and the return value will be of the type of the specified return type.

  Example:

```
int main()
{
   int a;

   printf("%d", a);

   return 0;
}
```

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.02

A.1 Aim:

Write a program to demonstrate all data types with:

- Variable declaration,
- Initialization
- Input/output statements.

A.2 Outcome:

After successful completion of this experiment students will be,

Demonstrate fundamental concepts of C Programming.

A.3 Theory:

Variable:

A variable is a name given to the memory location where data is stored depending upon the type of the variable.

Variable Declaration:

Syntax:

```
data_type variable_name;
```

Example:

```
int choice;
char ans;
```

Variable Initialization:

Variable initialization means assigning some value to that variable

Example:

```
int ch;
ch = 10;
```

OR

Variable Declaration and initialization on the same line

```
int ch = 10;
```

Data Type:

Data types refer to the characteristics of data stored into a variable.

Data type helps you find Size, range and type of value

C Data Types are used to:

Identify the type of a variable when it declared

Identify the type of the return value of a function

Identify the type of a parameter expected by a function

Data Type	Range	Bytes	Format
signed char	-128 to + 127	1	%c
unsigned char	0 to 255	1	%c
short signed int	-32768 to +32767	2	%d
short unsigned int	0 to 65535	2	%u
signed int	-32768 to +32767	2	%d
unsigned int	0 to 65535	2	%u
long signed int	-2147483648 to +2147483647	4	%ld
long unsigned int	0 to 4294967295	4	%lu
float	-3.4e38 to +3.4e38	4	%f
double	-1.7e308 to +1.7e308	8	%lf
long double	-1.7e4932 to +1.7e4932	10	%Lf
Note: The sizes and ranges of int, short and long are compiler dependent. Sizes in this figure are for 16-bit compiler.			

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.4 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.03

A.1 Aim:

To write a C program to find the largest of three numbers using nested if-else statements.

A.2 Outcome:

After successful completion of this experiment, students will be able to apply nested if-else statements and decision-making concepts in C programming to solve real-life problems.

A.3 Theory:

Decision Making:

Decision-making statements are used to control the flow of execution of a program based on specified conditions. In C, decision-making statements allow the program to execute different blocks of code depending on whether a condition is true or false.

if-else Statement:

The if-else statement in C is used to perform operations based on a specified condition. The statements inside the if block are executed when the condition is true; otherwise, the statements inside the else block are executed.

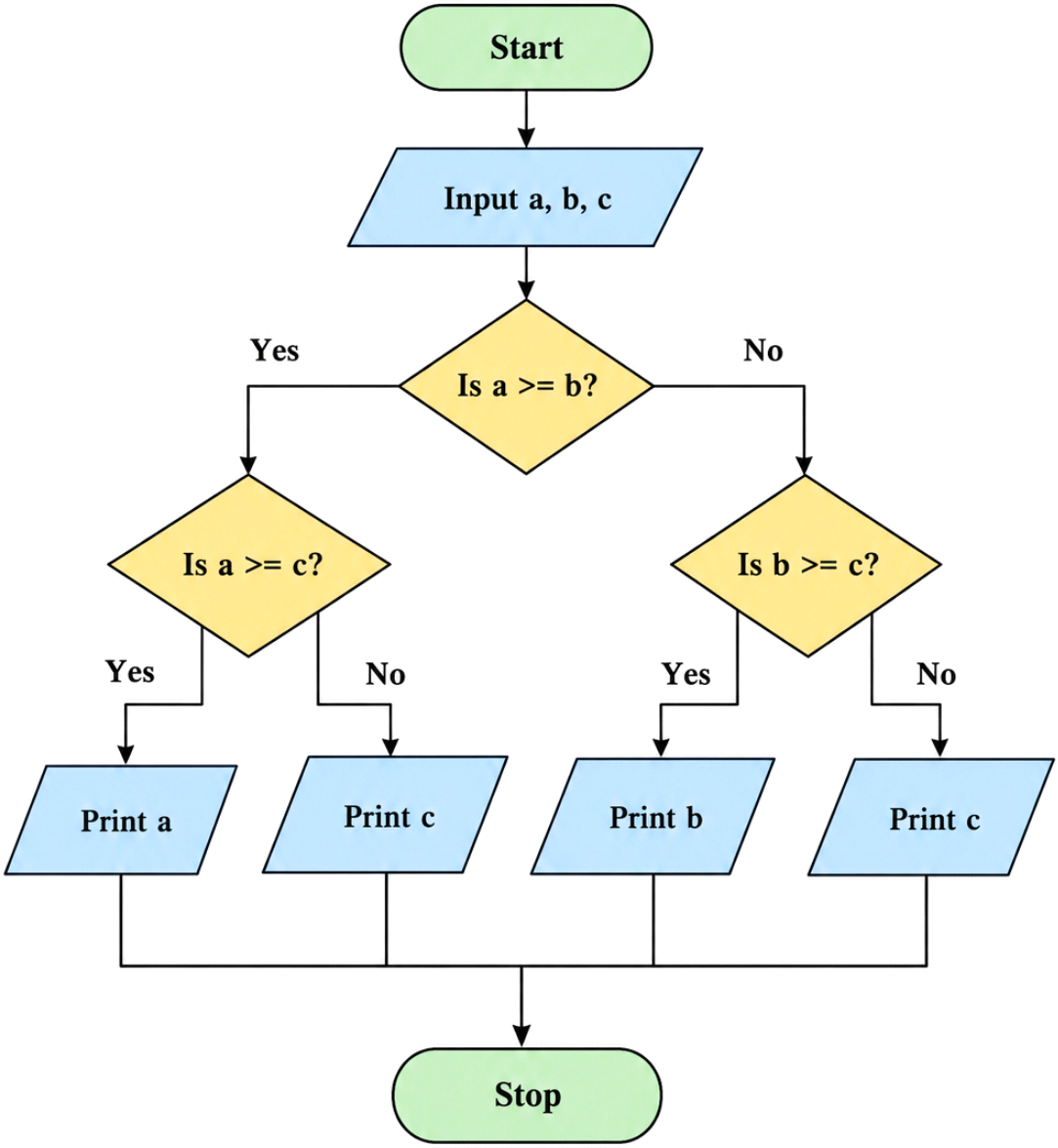
Nested if-else Statement:

A nested if-else statement is an if-else statement placed inside another if or else block. It is useful when multiple conditions must be checked to make a decision.

To find the largest of three numbers, the program compares the first number with the second and third numbers. If the first number is not the largest, the program compares the second and third numbers to determine the largest value.

Syntax:

```
if (condition1)
{
   if (condition2)
   {
      /* Statements */
   }
   else
   {
      /* Statements */
   }
}
else
{
   /* Statements */
}
```



Symbols Used:

- Start / Stop
- Input / Output
- Decision
- Process (Declaration)
- Flow Direction

A.4 Algorithm:

Step 1: Start.
Step 2: Declare three integer variables a, b, and c.
Step 3: Read three numbers from the user.
Step 4: Check whether a >= b.
If true check whether a >= c.
If true, display a as the largest number.
Otherwise, display c as the largest number.
Otherwise, check whether b >= c.
If true, display b as the largest number.
Otherwise, display c as the largest number.
Step 5: Stop.

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.4 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.04

A.1 Aim:

Given a year, determine whether it is a leap-year or not.

A.2 Outcome:

After successful completion of this experiment students will be,

Apply the concept of control structures for solving real life problems

A.3 Theory:

Decision Making:

Decision making is the most important aspect of almost all the programming languages. As the name implies, decision making allows us to run a particular block of code for a particular decision. Here, the decisions are made on the validity of the particular conditions. Condition checking is the backbone of decision making.

if else statement:

The if-else statement in C is used to perform the operations based on some specific condition. The operations specified in if block are executed if and only if the given condition is true.

Nested if else statement:

Nested If in C Programming is placing If Statement inside another IF Statement. Nested If in C is helpful if you want to check the condition inside a condtion. If Else Statement prints different statements based on the expression result (TRUE, FALSE). Sometimes we have to check even further when the condition is TRUE.

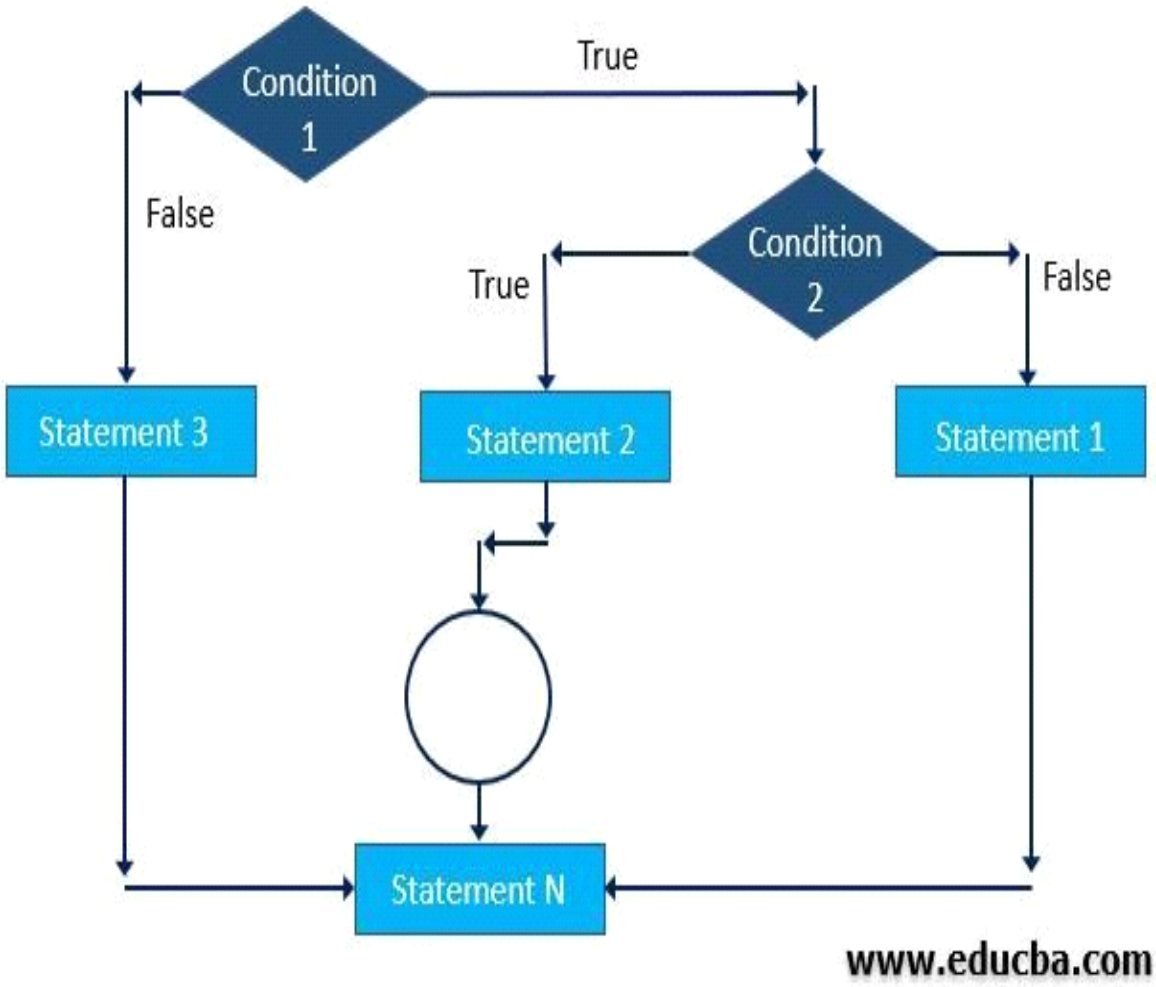
Syntax:

```
if ( test condition 1)
{

  //If the test condition 1 is TRUE then these it will check for test condition 2
  if ( test condition 2)
  {
   //If the test condition 2 is TRUE, these statements execute
   Test condition 2 True statements;
  }
```

```
   else
   {
    //If the c test condition 2 is FALSE, then these statements execute
    Test condition 2 False statements;
   }

else
{
 //If the test condition 1 is FALSE then these statements will be executed
 Test condition 1 False statements;
}
```



4. Algorithm:

Step 1: Start

Step 2: Read year

Step 3: if year>0 && year<=9999

if year%4 == 0 && (year%400==0 || year%100!=0)

Display year is a leap year

Else

Display year is not a leap year

Else

Display not a valid year

Step 4: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

Experiment No.05

A.1 Aim:

Menu-Driven code to implement app for billing system

A.2 Outcome:

After successful completion of this experiment students will be,

Apply the concept of control structures for solving real life problems

A.3 Theory:

Switch Case:

The switch statement in C is an alternate to if-else-if ladder statement which allows us to execute multiple operations for the different possible values of a single variable called switch variable. Switch statement in C tests the value of a variable and compares it with multiple cases. Once the case match is found, a block of statements associated with that particular case is executed. If a case match is NOT found, then the default statement is executed, and the control goes out of the switch block.

1) The switch expression must be of an integer or character type.

2) The case value must be an integer or character constant.

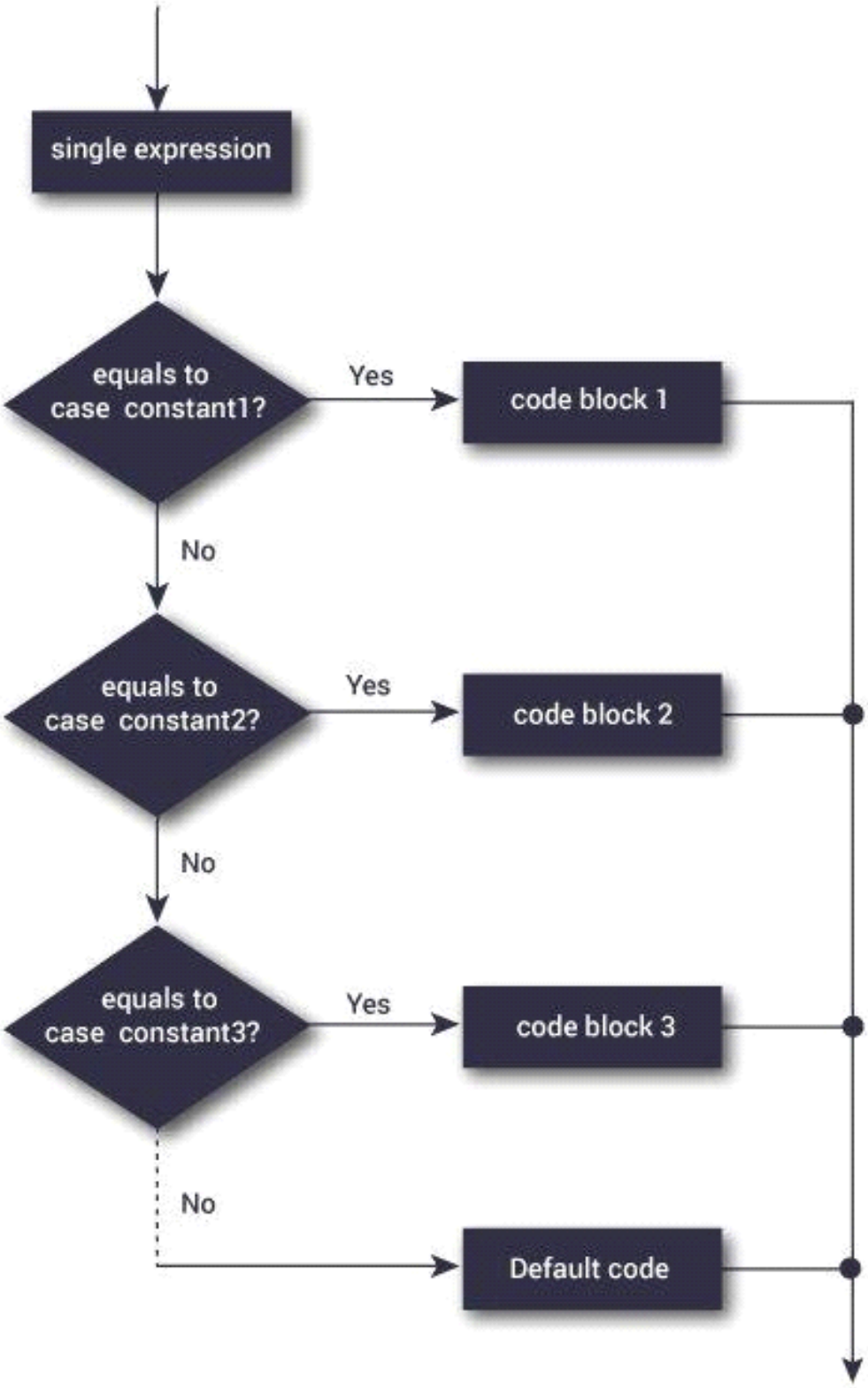
3) The case value can be used only inside the switch statement.

4) The break statement in switch case is not must. It is optional. If there is no break statement found in the case, all the cases will be executed present after the matched case. It is known as fall through the state of C switch statement.

Syntax:

```
switch(expression)
{
   case constant1:
    // statements
    break;

   case constant2:
    // statements
    break;
   .
   .
   .
   default:
        // default statements
}
```


single expression
equals to
case constant1?
Yes
code block 1
No
equals to
case constant2?
Yes
code block 2
No
equals to
case constant3?
Yes
code block 3
No
Default code

4. Algorithm:

Step 1**:** Start.

Step 2: Declare the variables choice, quantity, price, amount, and total.

Step 3: Initialize total = 0.

Step 4: Display the menu containing the available products and their prices.

Step 5: Accept the user's choice.

Step 6: Use the switch-case statement to select the product and assign its price.

Step 7: If the user selects an invalid option, display an error message and return to the menu.

Step 8: If the user selects a product, accept its quantity.

Step 9: Check whether the quantity is valid. If it is invalid, display an error message and return to the menu.

Step 10: Calculate the amount using the formula:

amount = price × quantity

Step 11: Add the item amount to the total bill:

total = total + amount

Step 12: Display the amount for the selected item.

Step 13: Repeat Steps 4–12 until the user selects the Generate Bill and Exit option.

Step 14: Display the final bill amount.

Step 15: Display the thank-you message.

Step 16: Stop.

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.06

A.1 Aim:

Write a C Program to print Star pattern - Triangle star pattern

- a)
```
*
* *
* * *
* * * *
* * * * *
```
b)
```
* * * * *
* * * *
* * *
* *
*
```

A.2 Outcome:

After successful completion of this experiment students will be,

Apply the concept of control structures for solving real life problems.

A.3 Theory:

Nested Loop:

C supports nesting of loops in C. Nesting of loops is the feature in C that allows the looping of statements inside another loop. Let's observe an example of nesting loops in C.

Any number of loops can be defined inside another loop, i.e., there is no restriction for defining any number of loops. The nesting level can be defined at n times. You can define any type of loop inside another loop; for example, you can define 'while' loop inside a 'for' loop.

Syntax:

```
Outer_loop
{
   Inner_loop
```

```
  {
      // inner loop statements.
  }
    // outer loop statements.
}
```

Nested For Loop:

The nested for loop means any type of loop which is defined inside the 'for' loop.

Syntax:

```
for (initialization; condition; update)
{
   for(initialization; condition; update)
   {
       // inner loop statements.
   }
   // outer loop statements.
}
```

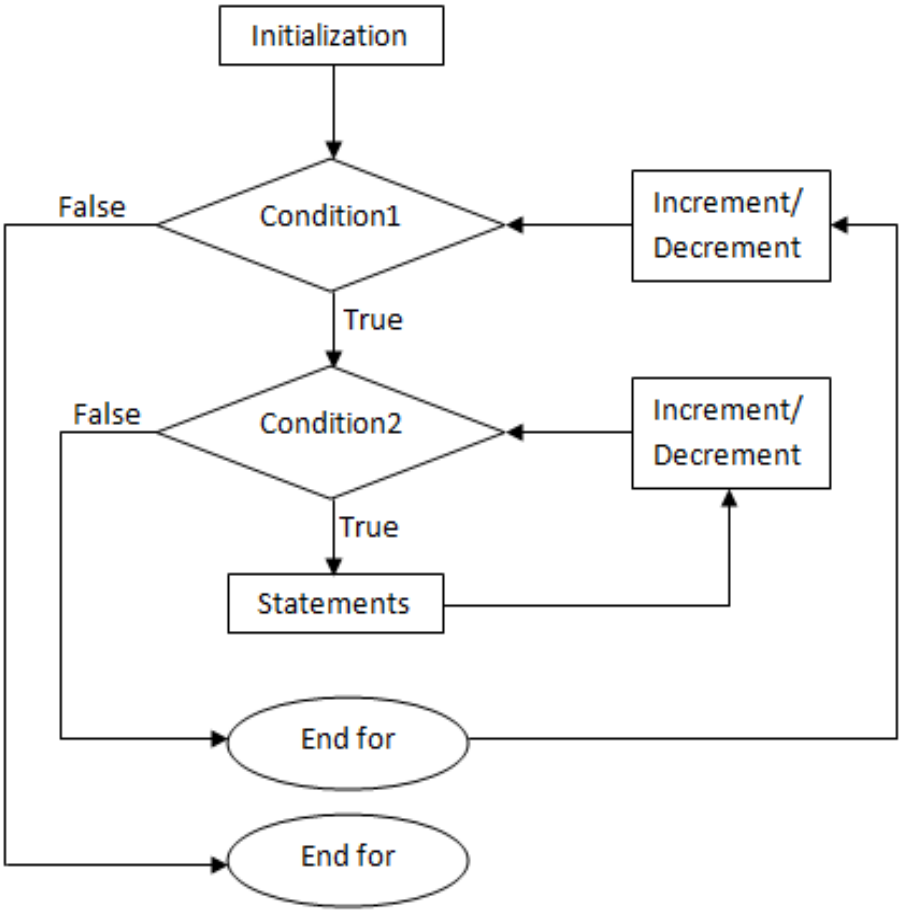


Fig: Flowchart for nested for loop

4. Algorithm:

a)

Step 1: Start

Step 2: for i = 1 to 5 do

for j = 1 to i do

Display *

end for loop

end for loop

Step 4: Stop

b)

Step 1: Start

Step 2: for i = 1 to 5 do

for j = i to 5 do

Display *

end for loop

end for loop

Step 4: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No. 9

A.1 Aim:

Write a C program to find multiplication of two matrix and transpose of the resultant matrix.

A.2 Outcome:

After successful completion of this experiment students will be,

Represent data in arrays, strings and structures and manipulate them through a program.

A.3 Theory:

Two Dimensional Array:

The two-dimensional array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns. However, 2D arrays are created to implement a relational database lookalike data structure. It provides ease of holding the bulk of data at once which can be passed to any number of functions wherever required.

Declaration of two dimensional Array:

Syntax:

```
data_type array_name[rows][columns];
```

Example:

```
int twodimen[4][3];
```

Here, 4 is the number of rows, and 3 is the number of columns.

Initialization of 2D Array in C

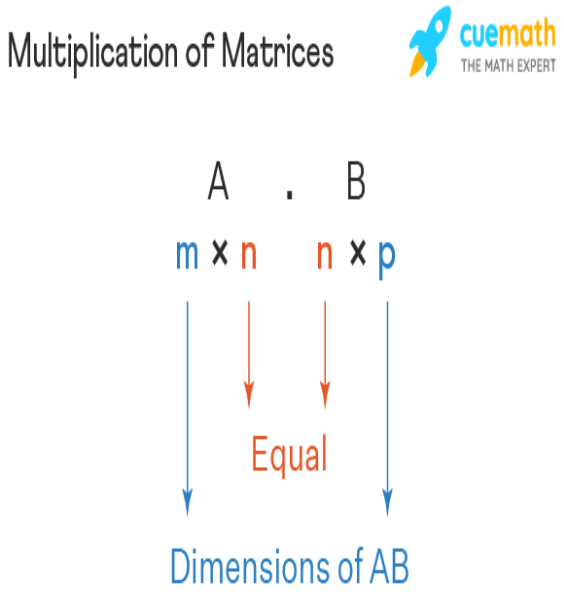
In the 1D array, we don't need to specify the size of the array if the declaration and initialization are being done simultaneously. However, this will not work with 2D arrays. We will have to define at least the second dimension of the array. The two-dimensional array can be declared and defined in the following way.

```
int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
```

	Column 1	Column 2	Column 3	Column 4
Row 1	x[0][0]	x[0][1]	x[0][2]	x[0][3]
Row 2	x[1][0]	x[1][1]	x[1][2]	x[1][3]
Row 3	x[2][0]	x[2][1]	x[2][2]	x[2][3]

Matrix Multiplication:

The multiplication of matrices is possible only when the matrices are compatible. Two matrices A and B are said to be compatible if the number of columns in A is equal to the number of rows in B. That means if A is a matrix of order m×n and B is a matrix of order n×p, then we can say that matrices A and B are compatible. That means, the resultant matrix for the multiplication of for any m × n matrix 'A' with an n × p matrix 'B', the result can be given as matrix 'C' of the order m × p.

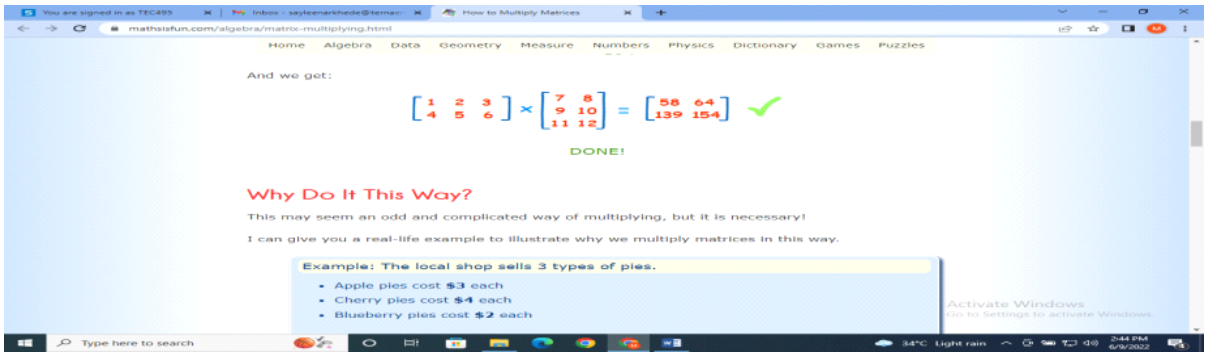


Multiplying matrices can be performed using the following steps:

Step 1: Make sure that the number of columns in the 1st matrix equals the number of rows in the 2nd matrix (compatibility of matrices).

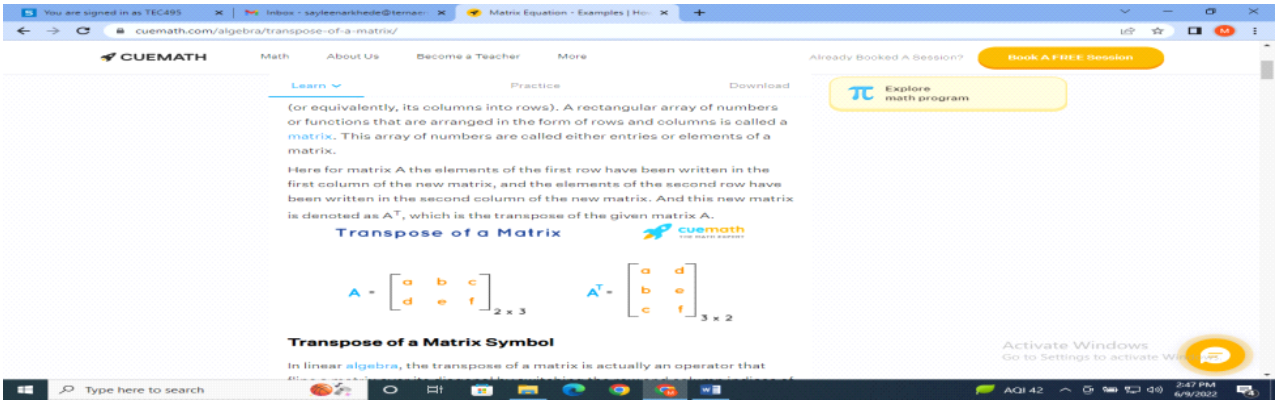
Step 2: Multiply the elements of ith row of the first matrix by the elements of jth column in the second matrix and add the products. This would the element that is in the ith row and j^{th} column of the resultant matrix.

Step 3: Place the added products in the respective positions.



Transpose of a Matrix:

The **transpose of a matrix** is obtained by changing its rows into columns (or equivalently, its columns into rows). Here for matrix A the elements of the first row have been written in the first column of the new matrix, and the elements of the second row have been written in the second column of the new matrix. And this new matrix is denoted as A^T, which is the transpose of the given matrix A.



4. Algorithm:

Matrix Multiplication:

Step 1: Start

Step 2: Read matrix a[m][n] and b[p][q]

Step 3: if n == p

Define matrix c[m][q]

for i = 0 to m-1

for j = 0 to q-1

c[i][j] = 0

for k = 0 to n-1

c[i][j] += a[i][k]*b[k][j]

Display matrix c[m][q]

else

Display matrix multiplication not possible

Step 4: Stop

Transpose of matrix:

Step 1: Start

Step 2: Define t[q][m]

Step 3: for i = 0 to m-1

for j = 0 to q-1

t[j][i] = c[i][j]

Step 4: Display t[q][m]

Step 5: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.10

A.1 Aim:

Write a C program for sorting numbers using Selection sort.

A.2 Outcome:

After successful completion of this experiment students will be,

Represent data in arrays, strings and structures and manipulate them through a program.

A.3 Theory:

Selection Sort:

• Selection sort is one of the easiest approaches to sorting.

• It is inspired from the way in which we sort things out in day to day life.

• It is an in-place sorting algorithm because it uses no auxiliary data structures while sorting.

How Selection Sort Works?

Consider the following elements are to be sorted in ascending order using selection sort-

6, 2, 11, 7, 5

Selection sort works as-

• It finds the first smallest element (2).

• It swaps it with the first element of the unordered list.

• It finds the second smallest element (5).

• It swaps it with the second element of the unordered list.

• Similarly, it continues to sort the given elements.

• As a result, sorted elements in ascending order are-

2, 5, 6, 7, 11

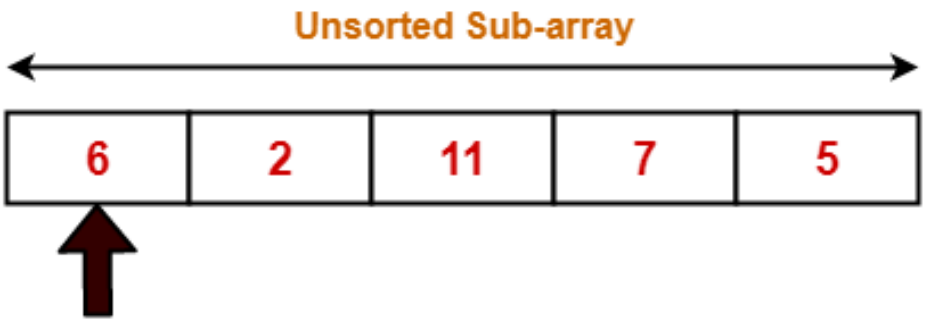
Selection Sort Example:

Consider the following elements are to be sorted in ascending order-

6, 2, 11, 7, 5

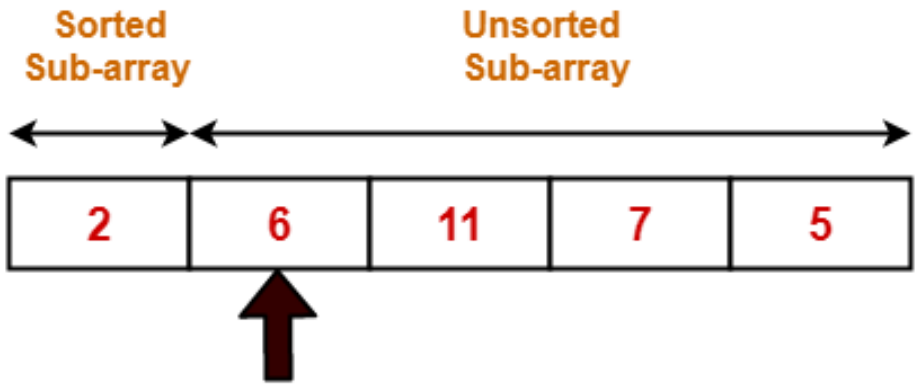
The above selection sort algorithm works as illustrated below-

Step-01: For i = 0



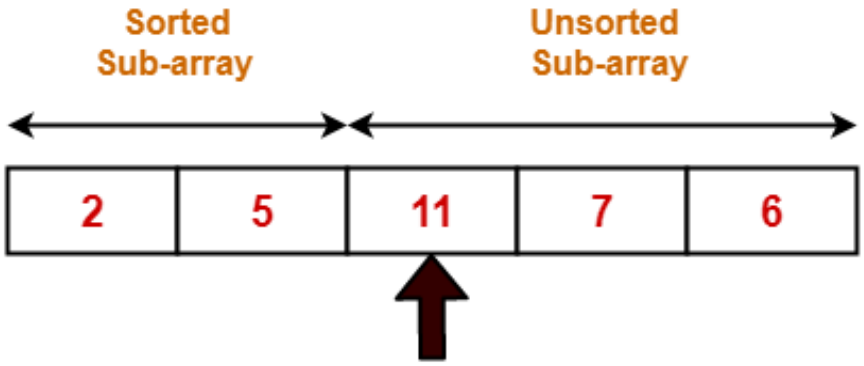
We start here, find the minimum element and swap it with the 1st element of array

Step-02: For i = 1



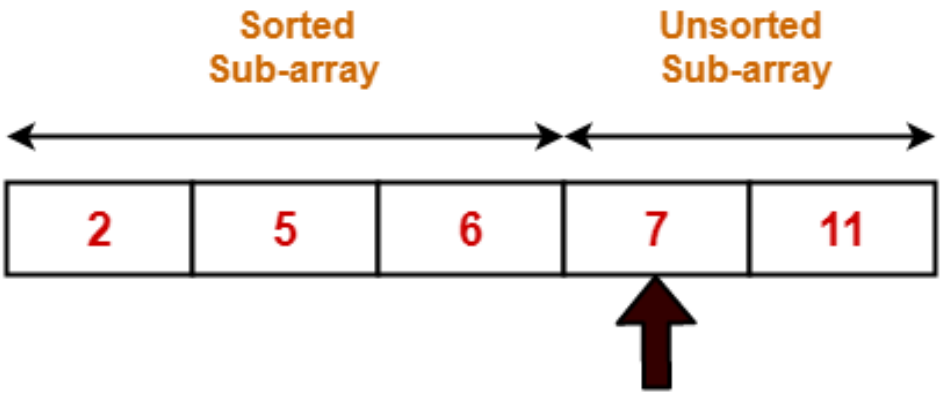
We start here, find the minimum element and swap it with the 2nd element of array

Step-03: For i = 2



We start here, find the minimum element and swap it with the 3rd element of array

Step-04: For i = 3

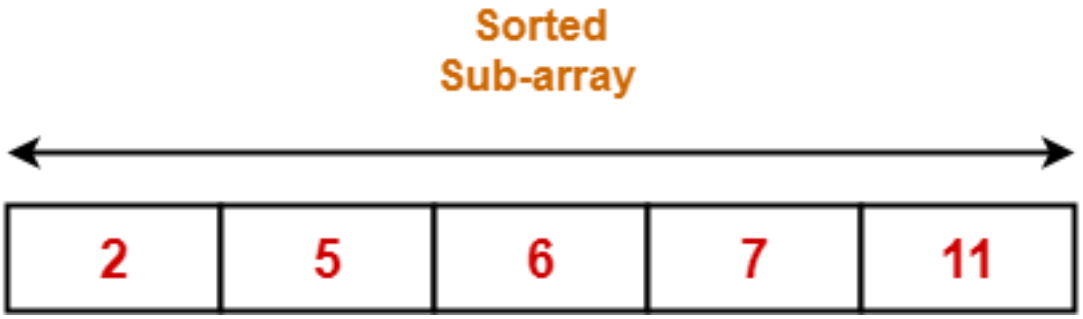


We start here, find the minimum element but there is no need to swap
(4th element is itself the minimum)

Step-05: For i = 4

Loop gets terminated as 'i' becomes 4.

The state of array after the loops are finished is as shown-



With each loop cycle,

The minimum element in unsorted sub-array is selected.
It is then placed at the correct location in the sorted sub-array until array A is completely sorted.

4. Algorithm:

Step 1: Start

Step 2: Declare array a of size n

Step 3: Set i = 0 and for each unsorted element in a repeat the steps 3 to 7

Step 4: Set min = i

Step 5: Set j = i+1 and repeat step 5

Step 6: if a[j] < a[min]

Set min = j

increment j

[End loop]

Step 7: Set temp = a[min]

Set a[min] = a[i]

Set a[i] = temp

Step 8: Increment i

Step 9: call Display()

Step 10: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.11

A.1 Aim:

Write a C Program to check whether string is palindrome or not.

A.2 Outcome:

After successful completion of this experiment students will be,

Represent data in arrays, strings and structures and manipulate them through a program.

A.3 Theory:

String:

The string can be defined as the one-dimensional array of characters terminated by a null ('\0'). The character array or the string is used to manipulate text such as word or sentences. Each character in the array occupies one byte of memory, and the last character must always be 0. The termination character ('\0') is important in a string since it is the only way to identify where the string ends. When we define a string as char s[10], the character s[10] is implicitly initialized with the null in the memory.

There are two ways to declare a string in c language.

- By char array

  char ch[10]={'j', 'a', 'v', 'a', 't', 'p', 'o', 'i', 'n', 't', '\0'};

- By string literal

  char ch[]="javatpoint";

  In this case, '\0' will be appended at the end of the string by the compiler.

Palindrome:

The characters in the string should remain the same after reversing the sequence of characters, the word should be read the same in both forward and backward then the string is known as "Palindrome".

Example:

"radar" word is a palindrome

4. Algorithm:

Step 1: Start

Step 2: Read str

Step 3: Initialize flag = 0

Step 4: call len = strlen(str) to find length of str

Step 5: for i = 0 to len

if str[i] != str[len-i-1]

flag = 1

break

Step 6: if flag == 1

Display strings are not palindrome

Else

Display strings are palindrome

Step 7: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.12

A.1 Aim:

Compare two strings without using library function.

A.2 Outcome:

After successful completion of this experiment students will be,

Represent data in arrays, strings and structures and manipulate them through a program.

A.3 Theory:

String Comparison:

Strings can be compared either by using the string function or without using string function.

The string function which is pre-defined in a **string.h** header file is a **strcmp()** function. The strcmp() function consider two strings as a parameter, and this function returns an integer value where the integer value can be **zero**, **positive** or **negative**. If ASCII value difference is zero strings are equal if it is –ve or +ve value other than 0 then strings are not equal.

The syntax of the strcmp() function is given below:

```
int strcmp (const char* str1, const char* str2);
```

In the above syntax, two parameters are passed as strings, i.e., **str1** and **str2**, and the return type is **int** means that the strcmp() returns an integer value.

The strcmp() function compares the character of both the strings. If the first character of both the strings are same, then this process of comparison will continue until all the characters are compared or the pointer points to the null character '\0'.

Possible return values from the strcmp() function

Return value	Description
0	When both the strings are equal.
<0	If the ASCII value of a character of the first string is less than the ASCII value of a character of the second string, then the function will return negative value.
>0	If the ASCII value of a character of the first string is greater than the ASCII value of a character of the second string, then the function will return positive value.

4. Algorithm:

Step 1: Start

Step 2: Read str1 and str2

Step 3: Initialize i = 0

Step 4: while s[i] != '\0' || d[i] != '\0'

if s[i] == d[i]

cmp = 0

i++

else

cmp = 1

break

Step 5: if cmp == 0

Display strings are equal

Else

Display strings are not equal

Step 6: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.13

A.1 Aim:

Find GCD of two numbers.

A.2 Outcome:

After successful completion of this experiment students will be,

Use modular programming approach for diversified problems.

A.3 Theory:

Function:

In c, we can divide a large program into the basic building blocks known as function. The function contains the set of programming statements enclosed by {}. A function can be called multiple times to provide reusability and modularity to the C program.

Function Aspects:

There are three aspects of a C function.

Function declaration: A function must be declared globally in a c program to tell the compiler about the function name, function parameters, and return type.

Syntax: return_type function_name (argument list);

Function call: Function can be called from anywhere in the program. The parameter list must not differ in function calling and function declaration. We must pass the same number of functions as it is declared in the function declaration.

Syntax: function_name (argument_list);

Function definition: It contains the actual statements which are to be executed. It is the most important aspect to which the control comes when the function is called. Here, we must notice that only one value can be returned from the function.

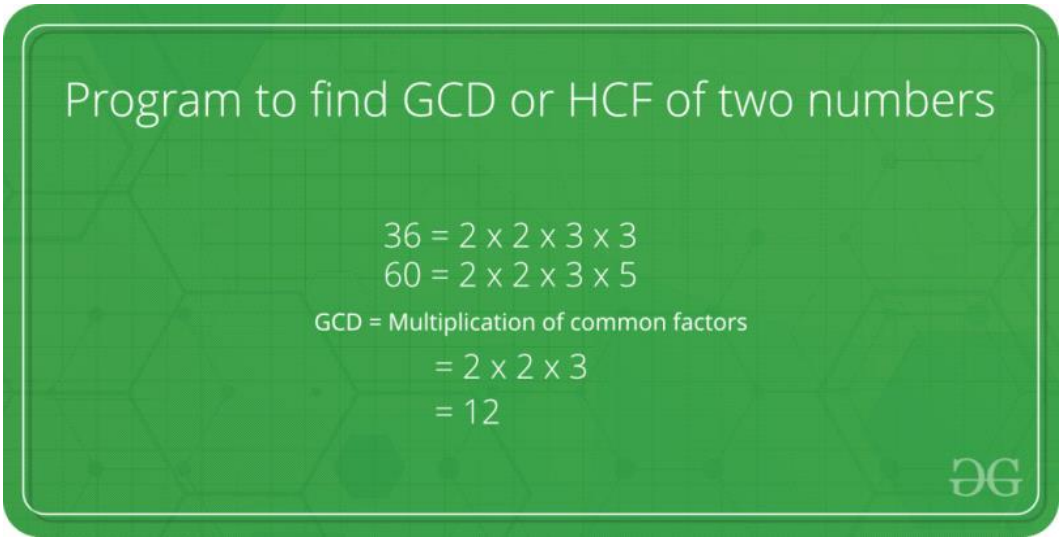
Syntax: return_type function_name (argument list) {function body;}

Return Value:

A C function may or may not return a value from the function. If you don't have to return any value from the function, use void for the return type.

GCD of two numbers:

The GCD is a mathematical term for the Greatest Common Divisor of two or more numbers. It is the Greatest common divisor that completely divides two or more numbers without leaving any remainder. Therefore, it is also known as the Highest Common Factor (HCF) of two numbers. For example, the GCD of two numbers, 20 and 28, is 4 because both 20 and 28 are completely divisible by 1, 2, 4 (the remainder is 0), and the largest positive number among the factors 1, 2, and 4 is 4. Similarly, the GCD of two numbers, 24 and 60, is 12.



4. Algorithm:

Main Function:

Step 1: Start

Step 2: Read n1 and n2

Step 3: call result = gcd(n1, n2)

Step 4: Display result

Step 5: Stop

gcd function:

Step 1: Start

Step 2: for i = 0 to n1 & n2

if n1%i == 0 && n2%i == 0

result = i

Step 3: return result

Step 4: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.14

A.1 Aim:

Search an element in the array using function.

A.2 Outcome:

After successful completion of this experiment students will be,

Represent data in arrays, strings and structures and manipulate them through a program.

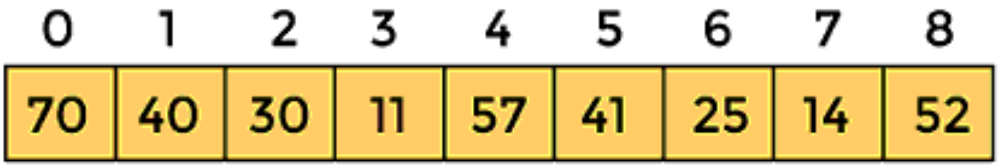
A.3 Theory:

Linear Search:

Linear search is a very simple search algorithm. In this type of search, a sequential search is made over all items one by one. Every item is checked and if a match is found then that particular item is returned, otherwise the search continues till the end of the data collection.

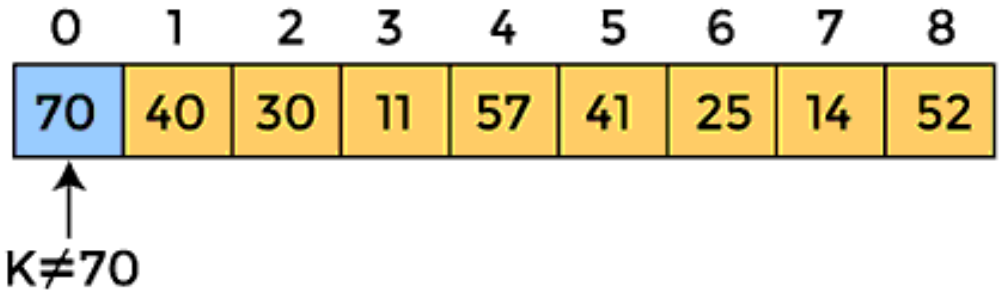
Working:

Let the elements of array are -

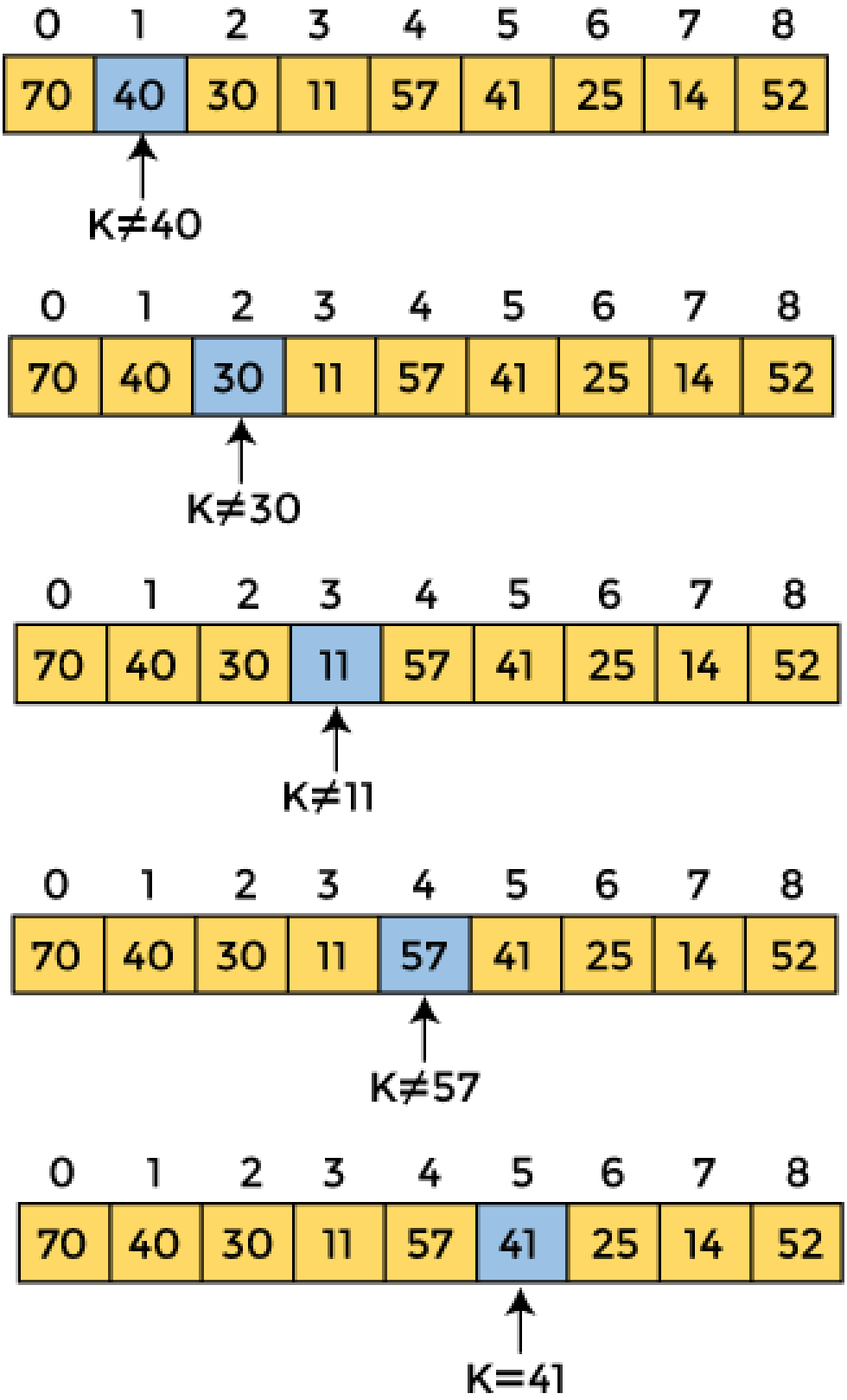


Let the element to be searched is **K = 41**

Now, start from the first element and compare **K** with each element of the array.



The value of **K,** i.e., **41,** is not matched with the first element of the array. So, move to the next element. And follow the same process until the respective element is found.


0 1 2 3 4 5 6 7 8
70 40 30 11 57 41 25 14 52
K≠40
0 1 2 3 4 5 6 7 8
70 40 30 11 57 41 25 14 52
K≠30
0 1 2 3 4 5 6 7 8
70 40 30 11 57 41 25 14 52
K≠11
0 1 2 3 4 5 6 7 8
70 40 30 11 57 41 25 14 52
K≠57
0 1 2 3 4 5 6 7 8
70 40 30 11 57 41 25 14 52
K=41

4. Algorithm:

Main Function:

Step 1: Start

Step 2: Read array arr[], key

Step 3: Call search(arr, key)

Step 3: Stop

search function:

Step 1: Start

Step 2: for i = 0 to n

if arr[i] == key

Display key is present in the array

break

Step 3: if i == n

Display key is not present in the array

Step 4: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.15

A.1 Aim:

Write a C program to print Fibonacci series of 20 numbers using recursion.

A.2 Outcome:

After successful completion of this experiment students will be,

Use modular programming approach for diversified problems.

A.3 Theory:

Recursion:

Recursion is the process which comes into existence when a function calls a copy of itself to work on a smaller problem. Any function which calls itself is called recursive function, and such function calls are called recursive calls. Recursion involves several numbers of recursive calls. However, it is important to impose a termination condition of recursion. Recursion code is shorter than iterative code however it is difficult to understand.

Recursive Function:

A recursive function performs the tasks by dividing it into the subtasks. There is a termination condition defined in the function which is satisfied by some specific subtask. After this, the recursion stops and the final result is returned from the function.

The case at which the function doesn't recur is called the base case whereas the instances where the function keeps calling itself to perform a subtask, is called the recursive case. All the recursive functions can be written using this format.

Each recursive call creates a new copy of that method in the memory. Once some data is returned by the method, the copy is removed from the memory. Since all the variables and other stuff declared inside function get stored in the stack, therefore a separate stack is maintained at each recursive call. Once the value is returned from the corresponding function, the stack gets destroyed. Recursion involves so much complexity in resolving and tracking the values at each recursive call. Therefore we need to maintain the stack and track the values of the variables defined in the stack.

Fibonacci Series:

Fibonacci series is a series of numbers formed by the addition of the preceding two numbers in the series. The first two terms are zero and one respectively. The terms after this are generated by simply adding the previous two terms.

4. Algorithm:

Main Function:

Step 1: Start

Step 2: Read n

Step 3: for i = 0 to n

f = fibo(i)

display f

i++

Step 4: Stop

fibo function:

Step 1: Start

Step 2: if n==0 || n==1

Return n

Else

Return fibo(n-1)+fibo(n-2)

Step 3: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.16

A.1 Aim:

Find product of digits of a number using recursion.

A.2 Outcome:

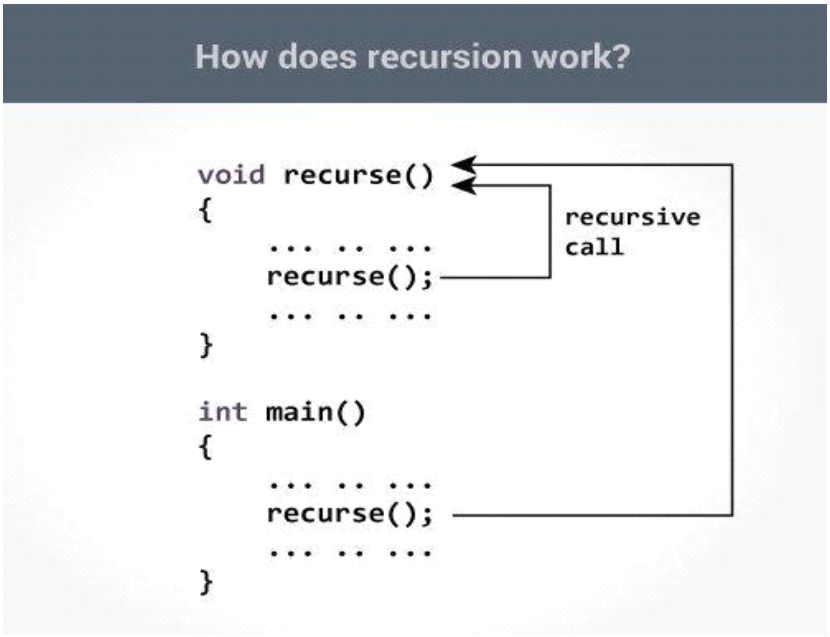
After successful completion of this experiment students will be,

Use modular programming approach for diversified problems.

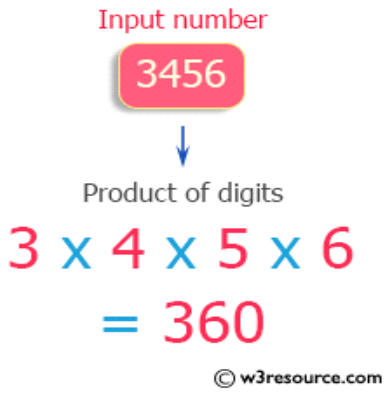
A.3 Theory:

Recursive Function:

A recursive function performs the tasks by dividing it into the subtasks. There is a termination condition defined in the function which is satisfied by some specific subtask. After this, the recursion stops and the final result is returned from the function.



Product of digits of number:



4. Algorithm:

Main Function:

Step 1: Start

Step 2: Read n

Step 3: product = pro(n)

Step 4: Display product

Step 5: Stop

pro function:

Step 1: Start

Step 2: if n<10

Return n

Else

Return n%10*pro(n/10)

Step 3: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.17

A.1 Aim:

Store product details in structure and calculate total amount.

A.2 Outcome:

After successful completion of this experiment students will be,

Represent data in arrays, strings and structures and manipulate them through a program.

A.3 Theory:

Structure in c is a user-defined data type that enables us to store the collection of different data types. Each element of a structure is called a member.

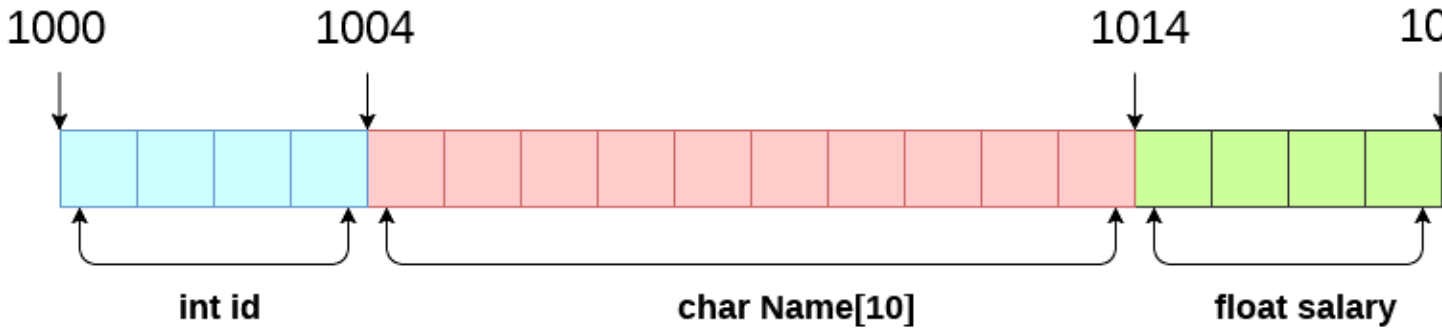
The struct keyword is used to define the structure.

Syntax:

```
struct structure_name
{
   data_type member1;
   data_type member2;
   .
   .
   data_type memeberN;
};
```

Structure Example:

```
struct employee
{   int id;
   char name[20];
   float salary;
};
```




```
struct Employee
{
    int id;
    char Name[10];
    float salary;
} emp;
```

```
sizeof (emp)  =  4 + 10 + 4 = 18 bytes

where;
 sizeof (int) = 4 byte
 sizeof (char) = 1 byte
 sizeof (float) = 4 byte
```

struct is the keyword; employee is the name of the structure; id, name, and salary are the members or fields of the structure.

Declaring structure variable:
We can declare a variable for the structure so that we can access the member of the structure easily. There are two ways to declare structure variable:

- ***By struct keyword within main() function***

```
struct employee
{   int id;
    char name[50];
    float salary;
};
struct employee e1, e2;
```

- ***By declaring a variable at the time of defining the structure.***

```
struct employee
{   int id;
    char name[50];
    float salary;
}e1,e2;
```

Accessing members of the structure:

By . (member or dot operator)
p1.id

4. Algorithm:

Step 1: Start

Step 2: Declare structure product with productID, product_name, price, quantity, total_Product_price

Step 3: Declare array of structure product[10]

Step 4: Rad n and Initialize total = 0

Step 4: for i = 0 to n

Read productID, product_name, price, quantity

total_product_price = price*quantity

Display productID, product_name, price, quantity, total_Product_price

Step 5: for i = 0 to n

total = total + total_product_price

Step 6: Display total

Step 7: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.18

A.1 Aim:

Demonstrate the example of nested structure.

A.2 Outcome:

After successful completion of this experiment students will be,

Represent data in arrays, strings and structures and manipulate them through a program.

A.3 Theory:

Nested Structure:

Nested structure in C is nothing but structure within structure. One structure can be declared inside other structure as we declare structure members inside a structure.

C provides us the feature of nesting one structure within another structure by using which, complex data types are created. For example, we may need to store the address of an entity employee in a structure. The attribute address may also have the subparts as street number, city, state, and pin code. Hence, to store the address of the employee, we need to store the address of the employee into a separate structure and nest the structure address into the structure employee.

The structure can be nested in the following ways.

- By separate structure
- By Embedded structure

1) Separate structure

Here, we create two structures, but the dependent structure should be used inside the main structure as a member.

```
struct Date
{
        int dd;
        int mm;
        int yyyy;
};
struct Employee
```

```
{
        int id;
        char name[20];
        struct Date doj;
}emp1;
```

2) Embedded structure

The embedded structure enables us to declare the structure inside the structure. Hence, it requires less line of codes but it cannot be used in multiple data structures.

```
struct Employee
{
        int id;
        char name[20];
        struct Date
        {
                int dd;
                int mm;
                int yyyy;
        }doj;
}emp1;
```

Accessing Nested Structure:

We can access the member of the nested structure by Outer_Structure.Nested_Structure.member as given below:

```
e1.doj.dd
e1.doj.mm
e1.doj.yyyy
```

4. Algorithm:

Step 1: Start

Step 2: Declare structure student with rollno, name, marks, structure address with plotno, building, pin

Step 3: Display Enter student record

Step 4: Read rollno, name, marks, plotno, building, pin

Step 5: Display rollno, name, marks, plotno, building, pin

Step 6: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.19

A.1 Aim:

Swap two numbers by call by reference.

A.2 Outcome:

After successful completion of this experiment students will be,

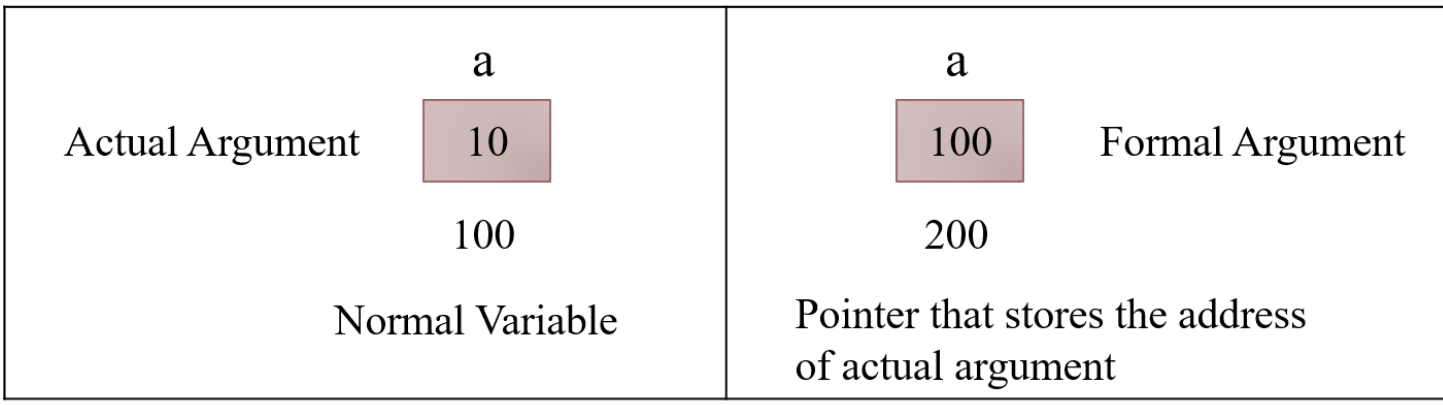
Use modular programming approach for diversified problems.

A.3 Theory:

Call by Reference:

In call by reference, the address of the variable is passed into the function call as the actual parameter. The value of the actual parameters can be modified by changing the formal parameters since the address of the actual parameters is passed.

In call by reference, the memory allocation is similar for both formal parameters and actual parameters. All the operations in the function are performed on the value stored at the address of the actual parameters, and the modified value gets stored at the same address.



4. Algorithm:

Main function

Step 1: Start.

Step 2: Display “enter any two numbers”

Step 3: Read a, b

Step 4: Display values of a and b before swap

Step 5: Call swap(&a,&b)

Step 6: Display values of a nd b after swap

Step 7: Stop

Swap function

Step 1: Start

Step 2: temp=*p

Step 3: *p=*q

Step 4: *q=temp

Step 5: Display a, b

Step 6: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion:

PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No.20

A.1 Aim:

Print a string using pointer.

A.2 Outcome:

After successful completion of this experiment students will be,

Demonstrate the concept of call by reference, dynamic memory allocation using pointers

A.3 Theory:

Pointer:

The pointer in C language is a variable which stores the address of another variable. This variable can be of type int, char, array, function, or any other pointer. The size of the pointer depends on the architecture.

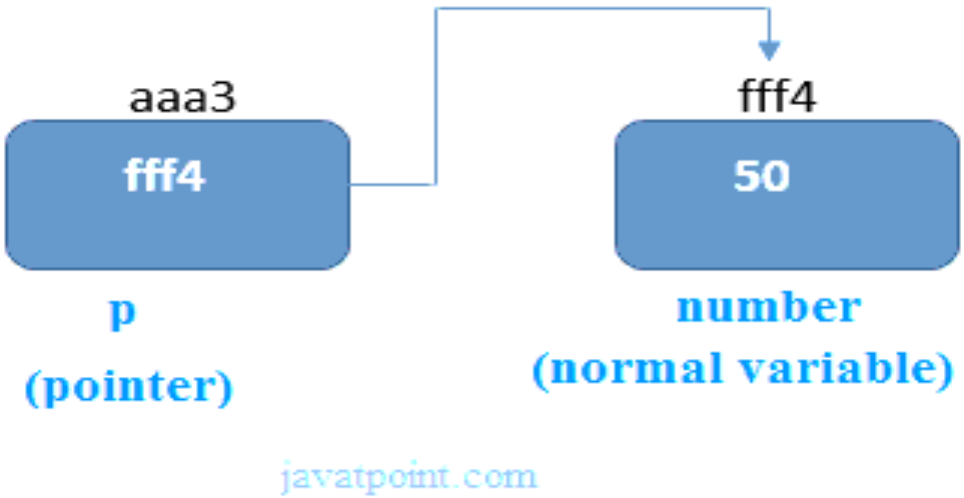
Consider the following example to define a pointer which stores the address of a character.

```
char n = 'a';
char* p = &n;   // Variable p of type pointer is pointing to the address of the variable n of t
ype character.
```

Declaring a pointer

The pointer in c language can be declared using * (asterisk symbol). It is also known as indirection pointer used to dereference a pointer.

```
int *a;//pointer to int
char *c;//pointer to char
```



String and Pointer:

When we create an array, the variable name points to the address of the first element of the array. The other way to put this is the variable name of the array points to its starting position in the memory.

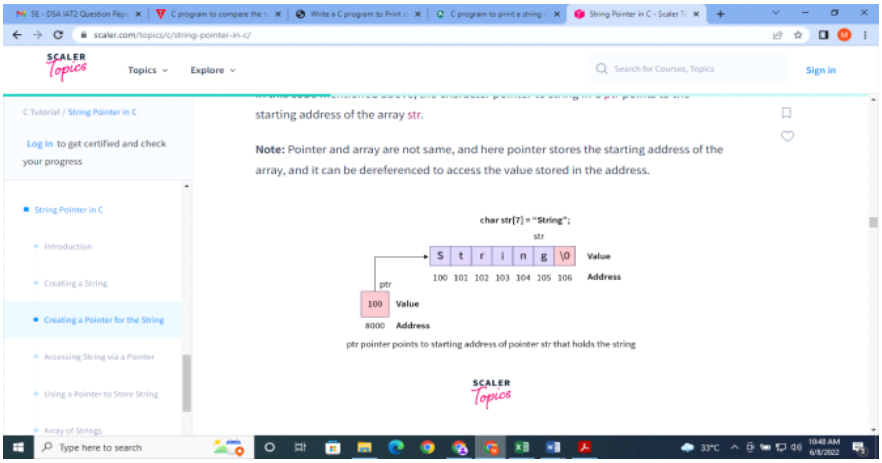
We can create a character pointer to string in C that points to the starting address of the character array. This pointer will point to the starting address of the string that is **the first character of the string**, and we can dereference the pointer to access the value of the string.

// charater array storing the string 'String'

char str[7] = "String";

// pointer storing the starting address of the character array str

char *ptr = str;



Accessing String via a Pointer

An array is a contiguous block of memory, and when pointer to string in C are used to point them, the pointer stores the starting address of the array. Similarly, when we point a char array to a pointer, we pass the base address of the array to the pointer. The pointer variable can be dereferenced using the asterisk * symbol in C to get the character stored in the address.

In this case, ptr points to the starting character in the array arr i.e. H. To get the value of the first character, we can use the * symbol, so the value of *ptr will be H. Similarly, to get the value of ith character, we can add i to the pointer ptr and dereference its value to get ith character.

Instead of incrementing the pointer manually to get the value of the string, we can use a simple fact that our string terminates with a null \0 character and use a while loop to increment pointer value and print each character till our pointer points to a null character.

4. Algorithm:

Step 1: Start.

Step 2: Read string str

Step 3: Declare *ptr and initialize ptr = str

Step 4: Initialize i = 0

Step 4: while *ptr != '\0'

Display *ptr

Increment ptr

Step 5: Stop

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.	Name:
Class	Batch:
Date of Experiment:	Date of Submission:
Grade:	

B.1 Code Screenshot:

B.2 Input and Output:

B.3 Conclusion: